STUDENT HANDOUT · DAY 5 · WEEKEND 3

Web Application Pentesting and Reconnaissance

Digital Forensics & Cyber Security Bootcamp · Atomcamp

Trainer: Waseem Abbas Khan — Senior Cybersecurity Specialist, CEI Master Trainer

Why this day is different

Days 1 to 4 attacked infrastructure. There was always an address, always a port, always a service that identified itself when asked. The target was a machine, and you could map it.

Today the target is a web application. It has one port — usually 443 — and behind that port sits an entire application with hundreds of endpoints, authentication logic, database queries, file uploads and business rules. Scanning the port tells you almost nothing. Everything interesting lives in the HTTP requests you send and the responses you receive.

This changes the discipline completely. The work becomes manual, hypothesis-driven and slow. The session split into two halves for that reason: the first half covers how a web application test is structured and which tools automate the boring parts; the second half covers reconnaissance, which is where most of the actual advantage is gained.

SCOPE AND LEGAL CONTEXT Reconnaissance sits at a boundary that matters legally. Passive reconnaissance — reading public records, search engines, certificate transparency logs — is normally lawful and touches nothing on the target's infrastructure. Active reconnaissance sends packets to the target and can be logged and attributed. This handout covers both, and the difference is stated at every step, because a student who does not understand it will eventually perform active scanning on something out of scope and have no defence.

Outcomes

#	Outcome	Evidence produced
1	Explain the client-server request/response model and the method vocabulary	An annotated example of one request and its response
2	Apply the three testing methodologies and state which phases they share	A phase map with three sources
3	Write a scope document and rules of engagement	A scope section that would survive legal review
4	Configure and run Burp Suite CE or OWASP ZAP for crawling and active scanning	A scan report from one of them
5	Construct Google dorks with the operators and explain what each one does	At least six working dorks against a target domain
6	Enumerate subdomains with three independent methods and reconcile the results	A deduplicated host list
7	Perform email reconnaissance and assess breach exposure	A list of addresses with exposure status
8	Describe the privacy operating systems and when they are appropriate	A written comparison

1. The Client-Server Model

Every web interaction is the same shape: a client sends a request, a server returns a response. Understanding that shape — and where an attacker modifies it — is the foundation for everything else in this day.

1.1 The request

```
GET /login.php HTTP/1.1
Host: target.example.com
User-Agent: Mozilla/5.0
Accept: text/html
Cookie: PHPSESSID=abc123
```

Component	Meaning	How an attacker abuses it
Method	The verb: GET, POST, PUT, DELETE, PATCH, OPTIONS, HEAD	Method tampering; verb-based access control bypass
Path	The resource requested	Path traversal, forced browsing, IDOR
Host header	Which site the client wants	Virtual host enumeration; cache poisoning; password-reset poisoning
User-Agent	The client software	Filtering-based bypass; mobile views with weaker controls
Cookie	Session and state	Session prediction, fixation, theft
Body	Data for POST, PUT, PATCH	Injection of every kind

1.2 The response

```
HTTP/1.1 200 OK
Server: Apache/2.4.49
Set-Cookie: PHPSESSID=abc123; path=/
Content-Type: text/html

<html>...
```

Three things in that response are immediate signals. The status code tells you the outcome category. The Server header states the software and version, which is a research lead. The Set-Cookie header without HttpOnly, Secure and SameSite attributes is a finding in itself, because it means the session cookie is readable by JavaScript and will travel over plain HTTP.

Status	Meaning	Why a tester cares
200	OK	The resource exists and is readable
301 / 302	Moved permanently / found	Follow them; redirects often reveal internal hostnames
401	Unauthenticated	Authentication is required — try removing the credential
403	Forbidden	Authenticated but not authorised — a bypass target, not a dead end
404	Not found	The baseline for forced browsing; note the response length, not just the code
500	Server error	Input reach an unexpected path; often the most informative code for a tester

THE STATUS CODE THAT MEANS THE MOST A 403 is not an obstacle, it is an invitation. It means the resource exists and the server is willing to tell you so. Authorization bypass — escaping the path, adding an extension, changing the case, using a different verb, supplying a header — is a standard part of web testing. A 404 tells you the resource is not there. A 403 tells you it is.

1.3 GET versus POST

	GET	POST
Data location	In the URL query string	In the request body
Bookmarkable	Yes	No
Cached by proxies	Often	Rarely
Length limit	Yes, practical	Effectively none

Security implication	Logged in server logs, browser history, Referer headers	Not logged in URLs, but still interceptable
Common mistake	Sensitive data in the query string	Assuming POST is encrypted or safe — it is not

POST is not a security control. It prevents data appearing in a URL, which is worth something, but every request passing through an intercepting proxy is fully readable regardless of method. This is the point at which an intercepting proxy becomes indispensable.

2. Testing Methodology

Three published methodologies were named in the session. They are not competitors; they describe the same engagement from different angles, and a professional document may cite all three.

2.1 The three frameworks

Framework	Owner	Character	Best for
PTES	Open community project	Seven high-level phases with a strong emphasis on scoping and reporting	Network and infrastructure engagements; a client who wants a familiar structure
OWASP WSTG	OWASP Foundation	Exhaustive, checkbox-driven catalogue of specific web tests	Web application engagements; ensuring nothing is missed
NIST SP 800-115	NIST (US government)	Formal, assessment-oriented, aligned with assurance programmes	Government, regulated and compliance-driven contexts

2.2 The PTES phases

The session began listing these and the notes left most of the detail blank. Each phase is completed here with what is actually produced.

Phase	What happens	Deliverable
1. Pre-engagement	Contract, scope, rules of engagement, authorisation signature	Signed authorisation; scope document
2. Intelligence gathering	Passive and active reconnaissance	Asset inventory; attack surface map
3. Threat modelling	Which assets matter, who would attack them, and how	Prioritised target list
4. Vulnerability analysis	Scanning, enumeration, manual review	Candidate vulnerability list
5. Exploitation	Controlled attempts to prove impact	Evidence of actual access
6. Post-exploitation	Privilege escalation, lateral movement, data access	Blast radius assessment
7. Reporting	Documentation of findings, severity, remediation	The deliverable

2.3 Black-box, grey-box and white-box, applied to web

Type	What you are given	The example used in the session	Realistic detection
Black-box	A domain name or IP only	The PSEB domain and its IP, no other information	Low; you test what an outsider can find
Grey-box	An IP plus portal access, or test logins	The portal with test credentials supplied	High; you reach authenticated functionality
White-box	Source code and architecture	The codebase itself, with the target already known	Certain; you read the flaw rather than guess it

THE SCOPE CONVERSATION THAT MATTERS MOST The grey-box case is the most valuable for web applications and the most commonly mis-sold. Testing an application from outside, with no credentials, tests the login page and little else. Testing with two test accounts — one for each role — reaches the authorization logic, which is where most serious

web flaws actually live. A client who buys a black-box web test and expects coverage of internal functions has bought the wrong thing, and it is your job to say so at scoping time.

2.4 Scope and rules of engagement

The session produced a scope example worth reproducing closely, because it is unusually well-formed.

Section	Example content	Why this element exists
In scope	*.intel.com; an IP address range or subnet; the APIs	Wildcards need a break-glass clause
Explicitly out of scope	career.intel.com	A named exclusion is unambiguous; 'everything else' is not
Prohibited techniques	No denial of service; no social engineering	Prevents an outage and a legal problem
Access granted	Test accounts, VPN, jump-server details	Determines the box classification
Testing window	Dates and hours	Production systems may not tolerate business-hours testing
Emergency contact	Named person, direct number, stop procedure	You will eventually need it

THE PROBLEM WITH WILDCARDS A wildcard scope such as *.intel.com is convenient and dangerous in equal measure. It includes every current subdomain, every subdomain created during the engagement, and every third-party service someone happens to point at that zone. It also includes career.intel.com, which is why the exclusion is written explicitly rather than left to interpretation. Every wildcard scope should carry a break-glass clause: a named contact who can authorise testing a new host, and a record of that authorisation.

2.5 Rules of engagement

Rule	Why it exists
Written authorisation before the first packet	Without it, testing is a criminal offence, not an assessment
Test only what is listed	Everything else belongs to someone else
Data minimisation	If you find customer records, prove access with one record and stop; do not copy the database
No destructive actions unless authorised	Deleting, encrypting or corrupting data is never implied by 'test it'
Stop on critical finding	Some clients require immediate notification rather than waiting for the report
Report promptly	A finding you sit on is a finding the client cannot act on
Destroy data afterwards	Confirms in writing when and how test data was destroyed

3. Automated Web Scanners

Automation handles discovery and the well-known checks. It does not handle logic, authorization or business rules. The professional use of a scanner is to clear the routine findings so that your manual time goes to the interesting ones.

3.1 The commercial and open-source landscape

Tool	Licence	Notes
Burp Suite Community	Free	Installed on Kali as burpsuite; intercepting proxy with limited scanner
Burp Suite Professional	Commercial	Full scanner, Intruder at full speed, extensions
OWASP ZAP	Open source (Apache 2.0)	Complete and free; installed as zaproxy
Netsparker / Invicti	Commercial	Proof-based scanning; reduces false positives
Acunetix	Commercial	Widely used in regional markets

THE COMMUNITY-EDITION LIMITATION In Burp Suite Community the scanner is present but deliberately limited — you can use the proxy, Repeater and Decoder fully, and you can run a scan, but the automated crawl-and-audit functionality is restricted. Do not conclude that the tool is broken. ZAP is the complete free alternative and is the right choice when a student needs full automated scanning without a licence.

3.2 OWASP ZAP

```
sudo apt install zaproxy -y
zaproxy
```

EXPECTED OUTPUT
(GUI opens; the traditional and modern interfaces are both present)

1. Set the target URL, then run Automated Scan. ZAP spiders the site and runs its active rules.
2. Alternatively, run the Manual Explore first, browsing the application through ZAP's proxy so that the site tree is populated, then attack the populated tree. This produces far better coverage than a blind spider and is what a professional does.
3. Add Marketplace add-ons for extra checkers, authentication handling and report formats.
4. Review the Alerts tab, sort by risk, and read each one rather than exporting blindly.

```
# ZAP can also run headless, which is what a pipeline uses
zaproxy -cmd -quickurl https://target.example.com -quickout /tmp/zap.html -quickprogress
```

WHY SPIDERS MISS MODERN APPLICATIONS Spidering follows links. Modern applications are single-page apps whose links are rendered by JavaScript, so a spider sees almost nothing. ZAP's AJAX spider and its browser integration exist for exactly this reason, and a modern engagement almost always needs them. If an automated scan of a single-page application returns only a handful of URLs, the scanner has not failed — it has done precisely what you asked, and you asked the wrong thing.

3.3 Burp Suite

1. Launch burpsuite. On first run, create a temporary project.
2. Configure the browser to use Burp as its proxy — 127.0.0.1:8080 — and install Burp's CA certificate so that HTTPS can be intercepted.
3. Browse the entire application manually. Every request enters the Proxy history and the site map, which becomes your application model.
4. Run Crawl and Audit against the site map. In the Community edition this is restricted.
5. For each interesting request, send it to Repeater and modify one thing at a time.

Installing the CA certificate is not optional. Without it, HTTPS requests appear in the proxy as encrypted noise, and the majority of modern applications are HTTPS-only.

```
export JAVA_HOME=/usr/lib/jvm/java-25-openjdk-amd64
burpsuite &
```

WHEN BURP WILL NOT START Burp is Java. If it will not launch, the first thing to check is the JDK — Java 25 is present on Kali 2025 and later, but a machine with an older or no JDK will fail silently or with a stack trace. Verify with 'java -version' before assuming the tool is at fault.

3.4 The manual workflow

The session listed the manual phases as recon, scan, enumerate, URLs, parameters, fuzzing, files and then injection classes. That sequence is correct, and each stage feeds the next.

Stage	Activity	What it produces
Recon	Map the application, technologies, hostnames	An asset inventory
Scan	Automated crawl and audit	Candidate findings
Enumerate	Directories, files, endpoints, parameters	A URL and parameter inventory
Fuzz	Send unexpected input to every parameter	Behavioural anomalies
Files	JavaScript, JSON, source maps, backups	Disclosed logic and secrets
Injection	SQLi, command injection, LFI/RFI, file upload	Confirmed vulnerabilities

The files stage deserves emphasis. JavaScript bundles frequently contain API endpoints, feature flags, internal hostnames and occasionally credentials. Source maps, when deployed, reconstruct the original source exactly. Backups — .bak, .old, .zip, .git — are routinely left in web roots. None of these require exploitation; they require looking.

4. Google Dorking

Search engines are reconnaissance tools. The operators are documented, public, and used legitimately by journalists, researchers and security professionals. What makes a dork effective is knowing which operator answers which question.

4.1 The operators

Operator	Function	Example	What it finds
intitle:	Matches the page title	intitle:"index of"	Open directory listings
inurl:	Matches the URL	inurl:admin	Administrative interfaces
intext:	Matches the body text	intext:"sql syntax near"	Applications leaking database errors
filetype:	Restricts to a file type	filetype:pdf	Documents at a given format
site:	Restricts to a domain	site:example.com	Everything indexed for one domain
ext:	Synonym for filetype	ext:env	Environment files with credentials
link:	Pages linking to a URL	link:example.com	Related and referring sites
cache:	The stored copy	cache:example.com	Pages that have since been removed
related:	Similar sites	related:example.com	Sibling and partner domains
"..."	Exact phrase	"internal use only"	Documents marked as confidential
-	Exclude a term	site:example.com -www	Subdomains only
OR / |	Either term	inurl:admin OR inurl:login	Multiple patterns at once

4.2 Applying them

The session used one dork as a demonstration and it is a good one, because it is harmless and it shows the mechanism:

```
intitle:"the hacker playbook" filetype:pdf
```

Two operators are combined: a title match and a file-type restriction. The result is a list of PDF files whose titles contain that phrase. Applied to a client domain, the same structure becomes a finding:

```
site:example.com filetype:pdf  "confidential"
site:example.com ext:env  "DB_PASSWORD"
site:example.com inurl:admin -inurl:login
site:example.com intitle:"index of"
site:example.com intext:"phpMyAdmin" OR intext:"phpinfo()"
```

A CONTROL YOU CAN RUN TODAY Dorking your own organisation is a legitimate, repeatable, free control that most organisations never perform. Run these queries against your employer's domain once a quarter, and report what is indexed. Everything you find is already public — an attacker does not need to break in to read it. This is one of the highest-value pieces of work a junior analyst can do, because it requires no special access and produces immediate, actionable findings.

Also worth knowing: Google's advanced search form exposes the same operators through a graphical interface, which is useful for demonstrating the technique to non-technical stakeholders. The session used a second example — "mr robot" site:drive.google.com — which illustrates that shared cloud drives are indexed and are a genuine data-exposure problem, not a curiosity.

4.3 Beyond Google

Resource	Use	Character
DuckDuckGo	Searching without personalisation	Passive; no tracking, different index
Exploit-DB	Public exploits and shellcode	Passive; the first place to check a version
Shodan	Internet-connected devices by banner	Passive; finds exposed services without touching them

Censys	Certificate and service data	Passive; excellent for host discovery
crt.sh	Certificate transparency logs	Passive; reveals subdomains from certificates
haveibeenpwned.com	Breach exposure by email	Passive; the standard breach check
intelx.io	Leaked data and dark-web index	Passive or commercial
Privacy Affairs / Research	Breach reporting and trends	Passive context

Shodan and Censys deserve a separate note. They are search engines for devices rather than pages: Shodan has already scanned the internet and indexed what answered, so searching it for a client's IP range reveals what an attacker can see without you sending a single packet to the target. That is passive reconnaissance in its purest form, and it is often the first thing a professional checks.

5. Web Footprinting

Footprinting answers a different question from port scanning. Not 'what is running' but 'what is this organisation running, and what does that tell me about the rest of it'.

5.1 Technology profiling

Tool	Where it runs	What it gives you
Wappalyzer	Browser extension	Technology stack of the page you are looking at, with versions
BuiltWith	Website	The same, plus historical records
whatweb	Kali command line	The same, scriptable, across many targets
WhatWeb	Also usable with aggression levels	Plugin enumeration, version fingerprints

```
whatweb https://example.com
whatweb -a 3 https://example.com      # aggressive: more plugins, slower
whatweb --color=never -a 1 example.com | grep -i "wordpress\|php\|jquery"
```

EXPECTED OUTPUT

```
https://example.com [200 OK] Apache[2.4.49], Country[US][US], HTML5,
 JQuery[3.5.1], PHP[7.4.3], WordPress[5.8.1], X-Powered-By[PHP/7.4.3]
```

That single line is a research plan. Apache 2.4.49 has a well-known path traversal vulnerability. PHP 7.4 is end of life. WordPress 5.8.1 is outdated and its plugin ecosystem is a common entry point. Each of those is a separate line of enquiry, and all of it came from one HTTP response.

5.2 WHOIS

```
whois google.com
whois example.com | grep -Ei "registrar|creation|expiry|name server|registrant"
```

Field	Why it matters
Registrar	Who holds the domain; relevant for takeover and dispute
Creation date	A domain registered last week is more likely to be malicious
Expiry date	An expired domain is a takeover target
Name servers	Reveals the DNS provider; a change here is a security event
Registrant	Often redacted now, but occasionally informative
Status flags	clientHold, serverHold and similar indicate the domain is suspended

For a domain registered under the .pk ccTLD, the authoritative source is PKNIC at pk6.pknic.net.pk rather than the generic WHOIS services. Regional registries are authoritative for their own zones, and the commercial aggregators such as whois.com and who.is simply query them — sometimes with stale caches.

5.3 Subdomain enumeration

Subdomain enumeration is where passive recon produces the largest return. A client's main website is usually hardened. Its dev subdomain, staging environment, or forgotten Jenkins instance is usually not.

Method	Tool	Mechanism	Coverage
Certificate transparency	crt.sh, subfinder (one source)	Every hostname that ever received a TLS certificate	Excellent; hard to avoid
Passive sources	subfinder -all, assetfinder	Search engines, DNS datasets, aggregators	Broad, dated
Search engine scraping	sublist3r	Multiple search engines in parallel	Broad
OSINT aggregation	theHarvester	Search engines plus PGP and other sources for hostnames and emails	Broad, plus email
Active DNS	amass, dnsenum, fierce	Zone transfer attempts, wordlist brute force	Complete for common names, noisy
Aggregation	amass	Combines passive and active sources	The most complete of the open-source set

```
subfinder -d example.com -silent
subfinder -d example.com -all -silent | sort -u > subs.txt
assetfinder --subs-only example.com | sort -u >> subs.txt
theHarvester -d example.com -b all -l 500
sublist3r -d example.com -o sublist3r.txt
amass enum -passive -d example.com -o amass.txt
```

WHY THREE TOOLS AND NOT ONE Run at least three. Each draws from different sources, and the union is larger than any single tool. The last step is always the same: concatenate, sort, deduplicate, then resolve. A list of candidate names is worthless until you know which of them are live, and the resolution step is where the httpx naming problem below becomes important.

5.4 Resolving the host list — and a collision to avoid

```
cat subs.txt | httpx -ip -td -sc -title | grep 200
```

EXPECTED OUTPUT

```
https://dev.example.com [200] [10.0.1.5] [Nginx, PHP] [Development Portal]
https://staging.example.com [200] [10.0.1.9] [Apache, WordPress] [Staging]
```

THE httpx NAME COLLISION A verified pitfall on Kali. The command above uses ProjectDiscovery's httpx, whose flags are -ip (resolve the address), -td (technology detect), -sc (status code) and -title. However, Kali also ships a Python library of the same name, and /usr/local/bin/httpx — the Python one — takes precedence over /usr/bin/httpx, which is the ProjectDiscovery toolkit installed from the httpx-toolkit package. Typing 'httpx' therefore runs the wrong program, and the flags above produce a usage error rather than results. Verify which one you are running with 'which httpx' and 'httpx -version', and invoke the toolkit by full path, /usr/bin/httpx, or check that /usr/local/bin comes later in PATH.

5.5 Email reconnaissance

Tool or service	Purpose	Notes
theHarvester	Harvest addresses and subdomains from public sources	Command line, free
hunter.io	Corporate email pattern discovery	Shows the pattern (first.last etc.) and named examples
recon-ng	A modular OSINT framework with database storage	The natural home for everything gathered in one engagement
haveibeenpwned.com	Breach exposure per address	The standard check; already-public data
intelx.io	Leak and paste-site index	Commercial beyond a limited free tier

```
recon-ng
# inside recon-ng
marketplace search
db init
workspaces create client_eng
marketplace install recon/domains-hosts/hackertarget
modules load recon/domains-hosts/hackertarget
options set SOURCE example.com
run
```

recon-ng is worth learning properly for one reason: it stores results in a database. An engagement generates hundreds of hostnames, addresses and contacts across dozens of tools, and a spreadsheet does not survive that. recon-ng gives you queries, reports and reproducibility.

5.6 A note on the data sources

The session listed a number of public and semi-public sources: PII aggregators, breach-compilation sites and forums. Two things need stating plainly.

1. Many of the named sites host material that is itself unlawful to possess or access in many jurisdictions, Pakistan included. Reading a breach forum is not the same as gathering a public record, and 'it was already leaked' is not a defence.

2. The professional equivalent is a licensed threat-intelligence service, a breach-notification service, or a client who explicitly supplies the data. If you need to know whether a client's credentials are exposed, haveibeenpwned answers that question, using only data that is already public by design.

A LINE WORTH DRAWING The distinction that keeps you out of trouble: checking whether a client's data has been exposed is authorised defensive work. Downloading the exposed dataset to see for yourself is not. Do the first, report what you found, and let the client obtain the dataset lawfully if they need it.

6. Privacy Operating Systems

These systems exist so that an analyst can conduct research without exposing their own identity or their client's infrastructure. They were listed in the session; this section explains what each actually does, because they solve different problems.

6.1 The comparison

System	Base	What it does	Best for	Limitation
Tails	Debian, live only	Routes everything through Tor; amnesic — nothing persists unless explicitly saved	Short research sessions from any machine	Cannot be used as a daily OS; slow
Whonix	Debian, two VMs	Gateway VM forces all traffic through Tor; workstation VM has no direct route	Sustained research where a leak would be catastrophic	Needs a hypervisor; higher resource cost
Qubes OS	Xen virtualisation	Compartmentalises every activity into a separate VM	High-assurance work; malware analysis	Hardware compatibility; heavy
Whonix on Qubes	Combination	Compartmentalisation plus enforced Tor	The strongest of the open-source options	Complex to set up

6.2 Why the design difference matters

Tails is amnesic: it runs from removable media, leaves no trace, and forgets everything. That is ideal for a single session and useless for ongoing work. Whonix is structural: the workstation VM literally has no network interface except the gateway, so even a compromised application cannot leak the real address. The protection survives a mistake in the application.

THE LIMIT OF ANY OF THEM None of these protect against a poorly constructed search. If you search for a unique string, log into a personal account, or enable JavaScript on an attacker-controlled page, your identity is compromised regardless of the operating system. Tor protects the transport, not the content of your conduct.

6.3 Practical alternatives

Need	Solution	Why
A Linux machine without installing	onworks.net	Runs a hosted Linux session in a browser
A preconfigured VM image	osboxes.org	Downloadable, configured images for VirtualBox and VMware
A disposable test host	A fresh VM with a snapshot	The simplest and most controllable answer
Browser isolation	A separate browser profile or VM	Prevents cookies and sessions crossing between engagements

7. Maltego and Link Analysis

Maltego is not a scanner. It is a link-analysis tool that models entities — domains, addresses, email accounts, organisations, people — and the relationships between them, and then automates the discovery of those relationships through transforms.

7.1 Core concepts

Concept	Meaning

Entity	A thing: a domain, an IP, an email address, a person, a phone number
Transform	A query that takes one entity and returns related ones
Graph	The accumulated result — the visual map
Case file	A saved graph, which is the deliverable for an OSINT phase

1. Register for Maltego CE at maltego.com — the Community Edition is free and sufficient for learning.

2. Launch maltego (installed on Kali) and sign in.

3. Create a new graph, drag a Domain entity onto it, and set the value.

4. Right-click and select Run Transform, then work through the discovery transforms.

5. Save the case file and export the graph as a PDF for the report.

WHAT IT IS ACTUALLY FOR The value of Maltego is not the transforms themselves — each one is a query you could write by hand. The value is the graph: when a hundred results are laid out as connected entities, an interrelationship becomes visible that no list would have shown. That is the same reason BloodHound matters in Active Directory work, and the reason an OSINT phase should end with a diagram rather than a spreadsheet.

8. Key Takeaways and Assignment

1. A web application has one port and endless surface. Port scanning tells you almost nothing.
2. A 403 means the resource exists. It is an invitation, not an obstacle.
3. POST is not a security control. Use an intercepting proxy or you are guessing.
4. Automation clears the routine findings. Authorization and business logic need a human.
5. Spiders cannot see a single-page application. Ask the AJAX spider or you will report a blank site as clean.
6. Grey-box is what a web test actually needs. A black-box test exercises the login page and little else.
7. The union of three subdomain tools is larger than any one. Always deduplicate, then resolve.
8. Checking whether data is exposed is authorised work. Downloading the exposed dataset is not.

Assignment

Task	Requirement	Weight
Scope document	Write a scope and rules-of-engagement section for a hypothetical *.intel.com engagement, including the exclusions, the break-glass clause and the emergency procedure	25%
Dork report	Find six exposures on a domain you are authorised to test (your own, your employer's, or a public domain you are not attacking) and document each dork, its result and the fix	25%
Subdomain reconciliation	Enumerate subdomains of one domain with three tools. Present a single deduplicated list, state which tool found what no other found, and explain why	25%
Scanner comparison	Run ZAP and Burp against the same target. Compare the findings: which did each find that the other missed, and why	25%

Dork the domain only if you own it or have written permission. Report findings without exploiting them.